

Bibliography Review (First Consultation Milestone)

Programme: MSc in Computer Science

Student: Hamed Mohamed (DG1EWF)

Thesis title: *Migrating Spring Boot MVC Systems to Java 21 Virtual Threads: Architecture, Risks, and Engineering Guidelines*

Supervisor: Morse Gregory Reynolds

Date: 2026-07-12

1. Scope and purpose of this bibliography

This bibliography review supports the first consultation milestone (“Review of the bibliography, 3–5 pages, approx. 10 sources”). The thesis investigates migration of legacy Spring Boot MVC applications to Java 21 Virtual Threads, with focus on two practical risks: **thread pinning** and **architecture-level anti-patterns** (for example synchronized blocks, problematic ThreadLocal usage, and blocking data access patterns).

The selected sources are intentionally mixed:

1. **Primary platform sources** (OpenJDK JEPs and Oracle docs) to ground the thesis in official Virtual Thread semantics.
2. **Framework/runtime sources** (Spring Boot, Tomcat, HikariCP) to connect language/runtime behavior with production Java web stacks.
3. **Measurement sources** (JFR and k6 documentation) to define reproducible methodology for pinning detection and benchmark interpretation.
4. **Foundational research papers** on thread-based vs event-based high-concurrency design to position Virtual Threads in broader concurrency literature.

Together, these sources define the thesis logic end-to-end: what Virtual Threads are, where migration fails in real applications, how to observe failures reliably, and how to derive engineering migration guidelines from measured evidence.

2. Thematic review

2.1 Virtual Threads and migration semantics

The central technical baseline is JEP 444, which finalizes Virtual Threads in Java 21 and clarifies their intended use as a drop-in scalability model for thread-per-request style servers. The key point for this thesis is that Virtual Threads increase concurrency capacity, but do not remove all contention or eliminate badly structured synchronization. This distinction directly motivates my research questions: migration success depends not only on enabling Virtual Threads, but also on code architecture and locking patterns.

JEP 425 and JEP 436 are also relevant because they document the preview-stage design rationale and behavior constraints before finalization. They help explain why some migration assumptions seen in older blog posts or early experiments can be outdated under JDK 21 final behavior. This is important for reproducibility and for accurate interpretation of benchmark outcomes in the thesis.

Oracle’s Java 21 Virtual Thread documentation complements the JEPs by translating semantics into operational guidance for developers. For my thesis, this source is used to align practical migration recommendations with official platform intent, especially around blocking code and scheduler behavior.

2.2 Spring Boot application-layer implications

Since the thesis target is Spring Boot MVC migration, Spring Boot’s official reference documentation is a primary source for framework-level support and configuration (`spring.threads.virtual.enabled`). This

source clarifies what the framework automates and, critically, what it does not. It does not rewrite application synchronization logic; therefore migration safety still depends on code audits and architecture refactoring.

Tomcat connector/threading documentation provides context for platform-thread baselines. In benchmark design, understanding standard Tomcat thread pool behavior is necessary for fair “before vs after” comparison. Without this baseline knowledge, a measured throughput change could be misattributed to Virtual Threads when it is actually caused by connector/thread-pool limits.

HikariCP documentation (including pool sizing guidance) is included because the data layer remains a hard constraint in both platform-thread and Virtual Thread modes. Even if server-side request handling can scale with Virtual Threads, database connection exhaustion can dominate latency and collapse throughput. This source supports my thesis argument that migration must consider resource bottlenecks beyond thread count.

2.3 Measurement and observability foundations

The thesis measurement strategy relies on two core observability sources:

- Java Flight Recorder (JFR) documentation for `jdk.VirtualThreadPinned` event collection and interpretation.
- Grafana k6 documentation for load generation and consistent latency/throughput/error metrics.

JFR is especially important because it provides a direct signal for pinning, which is otherwise easy to miss when only looking at endpoint latency. In my methodology, JFR allows causal analysis: not only “performance degraded,” but “performance degraded because specific call paths pinned virtual threads.”

k6 documentation is used to structure repeatable workload profiles (50/100/200 concurrency levels) and to keep test execution consistent across platform-thread baseline, naive Virtual Thread migration, and refactored Virtual Thread runs.

2.4 Concurrency literature context

To avoid a purely tool/documentation-driven thesis, I include foundational systems papers comparing threads and event-driven models under high concurrency. The von Behren et al. paper (“Why Events Are A Bad Idea...”) and the Welsh et al. paper (“SEDA”) provide classic but still relevant framing: scalability is an interaction between programming model, runtime scheduling, and backpressure/resource management.

Adya et al. adds useful theoretical perspective on asynchronous control-flow complexity and why developers prefer synchronous style despite scalability trade-offs. Virtual Threads can be interpreted as a modern JVM response to this historical tension: preserving synchronous code structure while targeting event-like scalability characteristics.

These papers strengthen the thesis contribution by placing Java 21 Virtual Threads in longer-term concurrency design evolution rather than treating them as a standalone language feature.

3. Source-by-source annotated bibliography

1. **OpenJDK.** (2023). *JEP 444: Virtual Threads*. <https://openjdk.org/jeps/444>
Relevance: Primary normative source for Java 21 Virtual Threads. Defines behavior, goals, and limitations; used as the formal basis for thesis terminology and migration assumptions.
2. **OpenJDK.** (2022). *JEP 425: Virtual Threads (Preview)*. <https://openjdk.org/jeps/425>
Relevance: Historical design context and early constraints; helpful when comparing earlier migration advice with Java 21 final behavior.
3. **OpenJDK.** (2023). *JEP 436: Virtual Threads (Second Preview)*. <https://openjdk.org/jeps/436>
Relevance: Transitional spec between first preview and finalization. Supports discussion of feature maturity and design adjustments before JDK 21.

4. **Oracle.** (2023). *Virtual Threads (Java Platform, Standard Edition 21 Documentation)*. <https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html>
Relevance: Practical platform guidance for developers; used to align migration recommendations with official JDK 21 operational guidance.
5. **Spring Team.** (Current). *Spring Boot Reference Documentation* (Virtual threads support section). <https://docs.spring.io/spring-boot/reference/features/task-execution-and-scheduling.html#features.task-execution-and-scheduling.virtual-threads>
Relevance: Framework integration layer for thesis experiments (`spring.threads.virtual.enabled`) and limits of framework-level automation.
6. **Apache Tomcat.** (Current). *HTTP Connector / Threading and request processing documentation*. <https://tomcat.apache.org/tomcat-10.1-doc/config/http.html>
Relevance: Baseline understanding of platform-thread request handling for fair comparative benchmarking and interpretation.
7. **HikariCP.** (Current). *Configuration and Pool Sizing Guidance*. Configuration knobs; About pool sizing
Relevance: Data-layer bottleneck analysis. Supports thesis claim that database pool sizing remains critical even after Virtual Thread adoption.
8. **Oracle.** (JDK 21). *jfr command documentation (Java Flight Recorder tool)*. Java 21 jfr man page
Relevance: Measurement validity for pinning analysis; defines collection and inspection of `jdk.VirtualThreadPinned` events.
9. **Grafana Labs.** (Current). *k6 Documentation*. <https://grafana.com/docs/k6/latest/>
Relevance: Load-testing methodology source for reproducible throughput, latency, and error-rate measurements in all three experiment modes.
10. **von Behren, R., Condit, J., & Brewer, E.** (2003). *Why Events Are A Bad Idea (for High-Concurrency Servers)*. In *HotOS IX*. <https://www.usenix.org/legacy/events/hotos03/tech/vonbehren.html>
Relevance: Foundational concurrency model comparison; informs thesis discussion of synchronous programming ergonomics versus scalability trade-offs.
11. **Welsh, M., Culler, D., & Brewer, E.** (2001). *SEDA: An Architecture for Well-Conditioned, Scalable Internet Services*. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. <https://doi.org/10.1145/502034.502057>
Relevance: Backpressure and staged resource management concepts relevant to interpreting bottlenecks in high-concurrency server systems.
12. **Adya, A., Howell, J., Theimer, M., Bolosky, W. J., & Douceur, J. R.** (2002). *Cooperative Task Management Without Manual Stack Management*. In *USENIX Annual Technical Conference*. <https://www.usenix.org/conference/2002-usenix-annual-technical-conference/cooperative-task-management-without-manual-stack>
Relevance: Classic discussion of asynchronous complexity; helps position Virtual Threads as a practical compromise between scalability and maintainable synchronous code.
13. **Project repository artifact.** (2026). *run-benchmark.sh* (benchmark orchestration with k6 + JFR). `run-benchmark.sh`
Relevance: Defines the exact local experimental procedure used in this thesis (startup, load execution, and JFR extraction).
14. **Project repository artifact.** (2026). *k6/load-test.js* (endpoint workload specification and thresholds). `k6/load-test.js`
Relevance: Primary source for workload shape and metrics collection rules in the reported measurements.
15. **Project repository artifacts.** (2026). *Raw benchmark outputs and JFR captures*. `results/*.json`; `results/jfr/*.jfr`

Relevance: Empirical evidence base for quantitative claims and pinning-event analysis in the thesis.

4. Preliminary gaps and next-step reading plan

The current set is sufficient for the first consultation milestone, but for the detailed research plan milestone I will extend it in three directions:

1. **Empirical Loom performance studies** in peer-reviewed venues (if available) to strengthen external validity claims.
2. **Database-driver and JDBC behavior under high Virtual Thread concurrency**, especially connection pool contention patterns.
3. **Case studies from production Java teams** migrating to Java 21 to compare thesis findings with industrial practice.

This next-step expansion will support tighter mapping between research questions, experiment design, and final methodological justification.